

AXONA

A Working Pub/Sub Roundtrip in Five Minutes

David A. Smith

Axona.net

The Axona Quick Start v4.48.0 2026-07-27

Axona Quick Start

Get a working pub/sub roundtrip in **under five minutes** on the current @axona/protocol **v4.48.0** API (kernel 4.48.0). One Node process connects to the live **testnet** bridge, subscribes to an open topic, publishes a signed message, and logs what comes back.

Two live networks, one line. Both run the 4.x kernel: **testnet** (`wss://testnet.axona.net`) tracks the newest release — 4.48.0, which this Quick Start pins — and **production** (`wss://bridge.axona.net`) runs the most recently promoted release. This Quick Start uses testnet; swap the bridge URL to target production.

Companion documents:

- Programmer Guide — the five ideas + recipes for real apps.
- API Reference — every exported symbol.
- AI Grounding — building with an AI assistant? Hand it this file.

Prerequisites

- **Node.js 20+** (for built-in Web Crypto Ed25519)
- A terminal with network access (we connect to `wss://testnet.axona.net`)

No build step, no DB, no local bridge — the testnet bridge is the entry point.

1. Install (30 seconds)

```
mkdir my-axona-demo && cd my-axona-demo
npm init -y
npm pkg set type=module
npm install github:axona-net/axona-protocol#v4.48.0
```

2. Two identities, one rule

Axona separates **who you connect as** from **who you publish as**:

- `createNodeIdentity({ lat, lng })` -> your **connection** identity. Its public key forms the 66-hex node ID (`.id`); the leading byte encodes your region. This authenticates the transport. It is fine to mint a fresh one each run.
- `createAuthorIdentity({ persistAs? })` -> your **author** identity. Its public key is the 64-hex Author ID (`.authorId`), which appears on the wire as `signerPubkey`. It has no location and no node ID — authorship is not a place. Pass `persistAs` to keep a stable author across runs.

The one rule: every publish names its signer via `{ signWith }` — an author identity, or the **ANONYMOUS** sentinel for a deliberately unsigned post. There is no default signer, and the node key never signs publishes.

`connect()` (below) mints both identities for you. To make the author *durable* across runs, pass `author: 'myapp:author'` — a `persistAs` key — instead of the ephemeral default. The underlying factories stay available for custom wiring.

3. Topics are descriptors, not strings

A topic is a small descriptor { region?, owner?, name, write? }; its 66-hex topic ID is a hash of that descriptor. Because the ID is a pure function of the fields, **anyone who knows the fields computes the same ID** — no registry, no coordination.

- region — a real geographic cell. Accepts a name ('useast'), a hex string ('0x89'), or a number (137) — all equivalent. Omit it and it defaults to the publisher's own node region; name it to share a topic across regions.
- owner — an Author ID, or absent.
- name — the human label ('lobby', 'profile').
- write — 'open' or 'owner'. **Defaults by owner:** no owner -> 'open' (anyone publishes); an owner -> 'owner' (only the owner key publishes, the safe default). With no owner, write is ignored — you can't have an owner-only topic with no owner.

This Quick Start uses an **open** topic { region, name }: anyone publishes, anyone subscribes. (Note: the removed v2 model — string topics, a publisher argument, sign:false, geoCellId/regionSynthPublisher synthetic publishers — is gone. Topics are descriptors now.)

deriveTopicId(descriptor) returns the 66-hex ID — the shareable read handle. sub accepts **either** a descriptor **or** that ID; pub always needs the descriptor (the storing node recomputes the ID to enforce the write policy).

4. Write the demo (one file)

Save this as index.js — it mirrors how apps/axona-minimal wires a peer:

```
import { connect, deriveTopicId, KERNEL_VERSION } from '@axona/protocol';

const BRIDGE = 'wss://testnet.axona.net';           // live testnet bridge (kernel 4.48.0)
const HERE   = { lat: 38.0, lng: -77.0 };           // your real location (us-east here)
const TOPIC  = { region: 'useast', name: 'quick-start-demo' }; // open topic

// 1-3. One call: mints both identities (connection + author), builds the
//      peer on the web transport, starts it, and waits for the mesh to warm.
console.log('kernel v' + KERNEL_VERSION + ' - connecting...');
const { peer, author, status, disconnect } = await connect({
  bridge: BRIDGE,
  location: HERE,
});
console.log('mesh ' + (status.ready ? 'ready' : 'not ready') + ' (' + status.peers + ' peers,
  ↪ ' + status.ms + 'ms)');
console.log('topic id:', await deriveTopicId(TOPIC));

// 4. Subscribe. The handler gets an envelope: { msgId, seq, ts, topic, message,
  ↪ signerPubkey? }.
// It fires for every delivery — including the ones we publish below — so you
```

```
// watch the counter climb live in the console.
const sub = await peer.sub(TOPIC, (env) => {
  console.log('[recv]', env.message, 'from', (env.signerPubkey || 'anon').slice(0, 12));
}, { since: 'all' });

// 5. Publish on a loop: one signed message per second, each carrying an
// incrementing counter, so the system is visibly, continuously running.
let n = 0;
const timer = setInterval(async () => {
  const msg = `tick #${++n}`;
  const msgId = await peer.pub(TOPIC, msg, { signWith: author });
  console.log('[pub ]', msg, '(msgId ' + msgId.slice(0, 12) + '...)');
}, 1000);

// 6. Keep running until Ctrl+C, then leave cleanly.
process.on('SIGINT', async () => {
  clearInterval(timer);
  await sub.stop();
  await disconnect(); // leave + stop, gracefully
  process.exit(0);
});
```

5. Run

node index.js

You should see the counter climb, one tick per second, until you stop it with **Ctrl+C**:

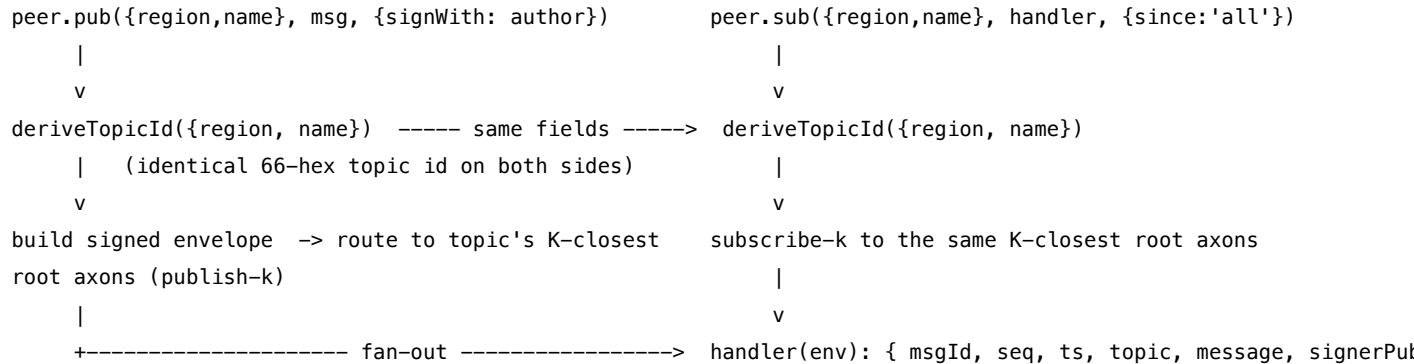
```
kernel v4.48.0 – connecting...
mesh ready (4 peers, 1200ms)
topic id: 89a1b2c3...
[pub ] tick #1 (msgId 8e9d4b1a30c2...)
[recv] tick #1 from 4f2a9b0c1d3e
[pub ] tick #2 (msgId 2c7f0a9b41d3...)
[recv] tick #2 from 4f2a9b0c1d3e
[pub ] tick #3 (msgId 91b3e5c8007a...)
[recv] tick #3 from 4f2a9b0c1d3e
...
```

That's it. You just:

- minted an Ed25519 connection identity (66-hex node ID, region-prefixed) and a location-free author identity (64-hex Author ID = signerPubkey),
- connected to the live bridge over the web transport and warmed up a routing mesh,
- derived a deterministic topic ID from { region, name },
- published a **signed** envelope to the topic's root axons and received it back through a since:

'all' subscriber.

What just happened



Both sides compute the identical topic ID because the ID is a pure hash of the descriptor fields. That ID-matching is the rule you can't break — same { region, name }, same topic.

Going further

Want to...	Do this
Make authorship stable across runs	<code>createAuthorIdentity({ persistAs: 'me' })</code>
Publish anonymously	<code>peer.pub(topic, msg, { signWith: ANONYMOUS })</code> (import ANONYMOUS)
Own a feed only you can write	<code>{ region, owner: me.authorId, name: 'profile' }</code> (write defaults to 'owner')
Share a read-only handle	<code>await deriveTopicId(descriptor) -></code> hand out the 66-hex ID; sub/pull/metrics accept it
Run against production	set <code>BRIDGE = 'wss://bridge.axona.net'</code> (same 4.x line; a separate network from testnet)
See the full mental model	Programmer Guide
Look up a specific symbol	API Reference

Troubleshooting

Cannot find module '@axona/protocol' — make sure `package.json` has `"type": "module"` and the install completed. Re-run `npm install`.

Cannot mix BigInt and other types — only applies to *manual* peer assembly on kernels before 4.14; `connect()` never hits it, and since 4.14 `NeuronNode` accepts the hex id directly.

peer.pub: name a signer... — `signWith` is required. Pass an author identity, or { `signWith: ANONYMOUS` } for an unsigned publish. There is no default signer.

Connects but nothing arrives — you almost certainly published before the mesh warmed up. Subscribing/publishing on a cold synaptome routes to too few root axons. `await peer.ready()` (step

3) before calling `pub/sub` — it resolves once the routing mesh is warm. On the public bridge this takes a few seconds.

Can't reach the bridge / connection hangs — confirm outbound `wss://` (TLS WebSocket) to `testnet.axona.net` is allowed by your network. There is no fallback to a local process; the demo needs the bridge to find peers.

UPGRADE_REQUIRED close code (4426) — a wire/version mismatch. The testnet bridge runs kernel 4.48.0 (wire 4.0); install `github:axona-net/axona-protocol#v4.48.0`. Production runs the same wire-4 line (one release behind), so 4426 there means a genuinely stale pin — upgrade to the bridge's kernel version (shown at its `/healthz`).